Part A

Ex: 4

Title: Program that performs Ordering of Directed Graphs

Problem Description: Program to obtain the Topological ordering of vertices in a given digraph with n vertices using source removal method

Method: A decrease and conquer approach to perform topological sort.

Theory Reference: Module 2

Algorithm

Iterate to Remove Source Vertices

- Create an array `removed[n]` initialized to `false` (0), tracking removed vertices.
- Repeat for `n` times:
  1. **Find a Source Vertex (In-degree 0)**
     - Find a vertex `j` such that:
       - `indegree[j] == 0` and
       - `removed[j] == false`
     - If no such vertex is found, **a cycle exists** → Stop and print "Graph is cyclic."
  2. **Print and Remove the Source Vertex**
     - Print vertex `j` (part of topological order).
     - Mark `removed[j] = true`.
     - Remove outgoing edges from vertex `j` by setting `cost[j][k] = 0` for all `k`.

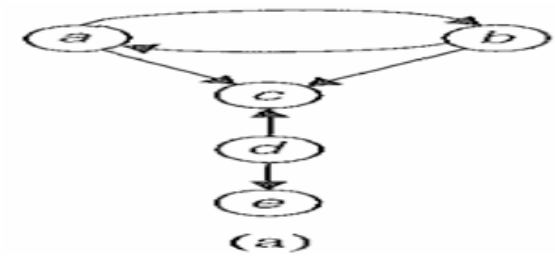
Program

```
import java.util.Scanner;

public class TopologicalSorting {

    static int[][] cost = new int[10][10];  // Adjacency matrix

    static int[] indegree = new int[10];    // Stores in-degree of nodes

    static int n;                           // Number of vertices

    // Function to calculate in-degree of all vertices

    static void calculate() {
```

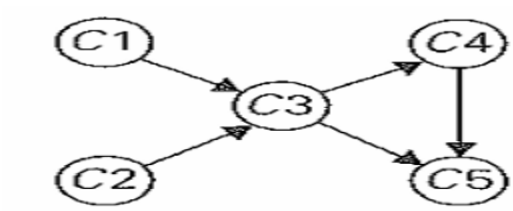
```
    for (int i = 0; i < n; i++) {
        indegree[i] = 0;
        for (int j = 0; j < n; j++) {
            indegree[i] += cost[j][i];  // Summing up column-wise to find in-degree
        }
    }
}
// Function to perform topological sorting using source removal method
static void sourceRemoval() {
    int[] removed = new int[10];
    System.out.print("Topological ordering is: ");
    for (int i = 0; i < n; i++) {
        calculate();  // Recalculate in-degrees after each removal
        int j;
        for (j = 0; j < n; j++) {
            if (removed[j] == 0 && indegree[j] == 0)  // Found a source vertex
                break;
        }
        if (j == n) {  // If no source is found, the graph has a cycle
            System.out.println("Graph is cyclic, so no solution.");
            return;
        }
```

```
            System.out.print(j + " ");
            removed[j] = 1;
            for (int k = 0; k < n; k++) {
                cost[j][k] = 0;
            }
        }
    }
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.print("Enter number of vertices: ");
        n = s.nextInt();
        System.out.println("Enter the cost matrix:");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                cost[i][j] = s.nextInt();
            }
        }
        sourceRemoval();
    }
}
```

Output




```
Enter number of vertices: 5
Enter the cost matrix:
0 1 1 0 0
1 0 1 0 0
0 0 0 0 0
0 0 1 0 1
0 0 0 0 0
Topological ordering is: 3 4 Graph is cyclic, so no solution.
BUILD SUCCESSFUL (total time: 53 seconds)
```




```
Enter number of vertices: 5
Enter the cost matrix:
0 0 1 0 0
0 0 1 0 0
0 0 0 0 0
0 0 0 0 1
0 0 0 0 0
Topological ordering is: 0 1 2 3 4 BUILD SUCCESSFUL (total time: 2 minutes 45 seconds)
```